내공 있는 프로그래머로 길러주는

컴파일러의 이해

Chapter 06
구문 분석

목차

03 상향식 구문 분석

04 모호한 문법의 사용과 에러 처리 루틴

학습목표

- 상향식 구문 분석 방법을 이해할 수 있다.
- 모호한 문법 사용과 에러 처리 루틴을 이해할 수 있다.

6.3 상향식 구문 분석

IT COOKBOOK

- 상향식 구문 분석은 주어진 문장이 문법에 맞는지 아닌지를 검사하기 위해 입력된 문자열을 읽어가면서 감축에 의해 시작 기호를 찾아가는 방법이다. 주어진 문자열이 시작 기호로 감축되면 올바른 문장이라고 판단하여 파스 트리를 생성하고, 그렇지 않으면 올바르지 않은 문장으로서 에러 메시지를 출력한다.
- **[정의 6-7] 감축과 핸들**
  - $S \overset{*}{\Rightarrow} \alpha Aw \Rightarrow \alpha\beta w$의 유도 과정이 존재할 때 문장 형태 $\alpha\beta w$에서 β를 A로 대체하는 것을 감축(reduce)이라 하고, β를 문장 형태 $\alpha\beta w$의 핸들(handle)이라 한다. 즉 한 문장 형태에서 감축되는 부분이 핸들이다.

6.3 상향식 구문 분석

IT COOKBOOK

[정의 6-8] 핸들 대체

- $S \underset{rm}{\Rightarrow} \omega_1 \underset{rm}{\Rightarrow} \omega_2 \underset{rm}{\Rightarrow} \cdots \underset{rm}{\Rightarrow} \omega_{n-1} \underset{rm}{\Rightarrow} \omega_n (=\omega)$의 유도 과정이 있을 때 각 ω_i의 문장 형태
- 에서 핸들을 찾아 ω_{i-1} 로 가면서 시작 심벌로 감축되는 과정을 핸들 대체(handle pruning)라고 한다.

이동-감축(shift-reduce) 구문 분석

- 이동-감축 구문분석은 상향식 방법의 일종
- 스택(stack)과 입력 버퍼(input buffer)를 사용하여 구현
  - 스택은 보통 파싱 스택(parsing stack)이라 부르며 문장 형태에서 핸들을 찾을 때까지 필요한 문법 심벌들을 유지하고 입력 버퍼는 주어진 문자열을 보유
- 이동-감축 구문분석기

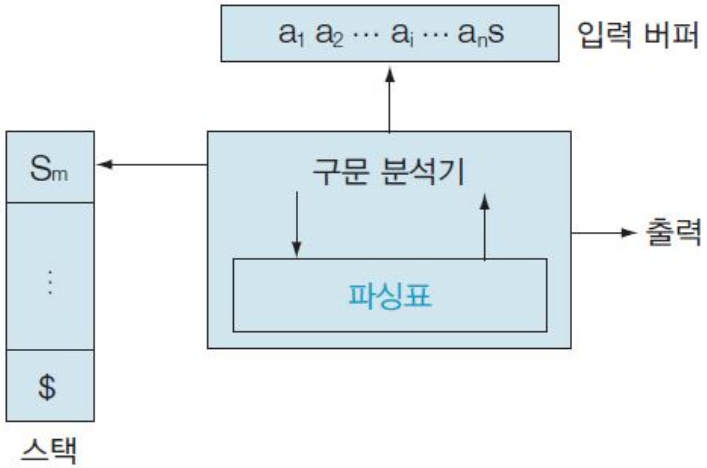

그림 6-3 이동-감축 구문 분석기의 구성

6.3 상향식 구문 분석

IT COOKBOOK

▪ 이동-감축 구문분석기의 행동

- 이동(Shift) : 현재의 입력 기호를 스택의 톱에 옮기는 것을 의미
  - 이 과정은 스택의 톱에 핸들이 나타날 때까지 계속
- 감축(Reduce) : 핸들이 스택의 톱에 나타나면 스택의 톱이 핸들의 오른쪽 끝이 되고, 핸들의 왼쪽 끝을 찾아서 완전한 핸들을 찾은 다음 핸들에 해당되는 생성 규칙의 왼쪽에 있는 기호로 대체되는 것
- 수락(Accept) : 주어진 문자열이 주어진 문법에 맞는 문장이라는 것을 나타냄.
- 에러(Error) : 주어진 문장이 주어진 문법에 맞지 않는 문장임을 의미하고 에러 처리 호출한다.

6.3 상향식 구문 분석

IT COOKBOOK

- 다음의 이동 -감축 구문 분석 과정을 살펴보자.

① 입력 문자열을 읽고 핸들을 발견한다.

② 핸들을 찾으면 감축하고 부분 파스 트리를 구성한다.

③ 같은 방법으로 계속 진행한다.

④ 입력 문자열을 모두 읽은 후 시작 기호를 만나면 주어진 문장은 맞는 문장이 된다

6.3 상향식 구문 분석

■ 연산자 우선순위 구문 분석

- 연산자 우선순위 구문 분석(operator precedence parsing)은 연산자 상호 간의 우선순위 관계(operator precedence relation)에 의해 핸들을 찾는 방법으로 연산자 우선순위 문법이 필요하다.

■ [정의 6-9] 연산자 우선 순위 문법

- 연산자 우선순위 문법(operator precedence grammar)은 연산자 문법이면서 2개의 터미널 기호 사이에 많아야 1개의 연산자 우선순위 관계를 갖는 것을 말한다. 이 문법에 의해 정의된 언어를 연산자 우선순위 언어라고 한다.

- 어떤 문법이 주어졌을 때 그 문법이 연산자 우선순위 문법인지 바로 알 수가 없다. 연산자 우선순위 관계는 연산자 우선순위 파싱표를 만들어봐야 알 수 있기 때문이다.

6.3 상향식 구문 분석

IT COOKBOOK

- 연산자 우선순위 구문 분석 방법은 연산자 우선순위 문법이 주어졌을 때 생성 규칙의 문법 기호 상호 간의 연산자 우선순위 관계에 의해 핸들을 찾는 방법으로 상향식 구문 분석 방법이다. 이 방법은 파싱표를 작성하기가 쉽고 연산자 간에 우선순위를 정하여 구문 분석을 할 수 있어 산술식을 위한 구문 분석 방법으로 아주 적당하다. 반면에 구문 분석되는 문법의 범위가 작고 터미널과 터미널만의 우선순위 관계에 의해 핸들을 찾으므로 구문 분석이 완전하게 되지 않는다는 단점이 있다. 그럼에도 불구하고 단순성 때문에 연산자 우선순위 구문 분석기를 사용한 컴파일러가 많이 만들어졌다.
- 연산자 우선순위 구문 분석기는 연산자 우선순위 관계를 가진 연산자 우선순위 문법을 사용한다. 연산자 우선순위 구문 분석이 정확히 이뤄지도록 연산자 우선순위 관계를 만드는 여러 가지 방법이 있다. [예제 6-20]의 문법에 대해 적합한 우선순위 관계를 얻기 위해 다음과 같은 경험적 방법을 사용해보자. [예제 6-20]에 주어진 문법은 모호한 문법이다. 다음 규칙은 이항 연산자에 대해 결합 법칙과 우선순위를 반영하여 적절한 핸들을 선택하게 한 것이다.

6.3 상향식 구문 분석

IT COOKBOOK

- ① 만약 연산자 θ1이 연산자 θ2보다 우선순위가 높다면 θ1 ⋗ θ2 그리고 θ2 ⋖ θ1이다. 예를 들어 *가 +보다 높은 우선순위일 때 * ⋗+ 그리고 + ⋖ *이다. 이러한 관계를 통해 식 E + E * E + E에서 가운데에 있는 E * E가 가장 먼저 감축될 핸들임을 알 수 있다.
- ② 만약 연산자 θ1과 θ2가 동일한 우선순위이고 연산자들이 왼쪽 결합 법칙을 가지면 θ1 ⋗ θ2 그리고 θ2 ⋗ θ1이다. 또한 연산자들이 오른쪽 결합 법칙을 가지면 θ1 ⋖ θ2 그리고 θ2 ⋖ θ1이다. 예를 들어 +가 왼쪽 결합 법칙을 가지면 + ⋗ +이다.
- ③ 모든 연산자 θ에 대해 θ ⋖ id, id ⋗ θ, θ ⋖ (, (⋖ θ,) ⋗ θ, θ ⋗ $, $ ⋖ θ이다. 또한 다음과 같이 되게 한다.

(≐)	$ ⋖ (	$ ⋖ id
(⋖ (	id ⋗ $	) ⋗ $
(⋖ id	id ⋗)	) ⋗)

- 이 규칙은 id와 (E)를 모두 E로 축소시킨다. 또한 $와 $ 사이 어디에서나 핸들이 발견되어야 하므로 $는 스택의 톱과 입력 버퍼 오른쪽 끝의 표시(right endmarker)로 사용된다.

6.3 상향식 구문 분석

IT COOKBOOK

LR 구문 분석

- 결정적인 상향식 구문 분석 방법
- LR (Left to right scanning and Right parse)은 입력 문자열을 왼쪽에서 오른쪽으로 읽어가며, 출력으로 우파스를 생성
- LR 방법으로 구문 분석을 행하는 구문분석기 : LR parser

주요 내용

- 주어진 문법으로부터 파싱 테이블을 구성하는 이론과 방법
- 파싱 테이블이 주어졌을 때, LR 파서가 어떻게 동작하는가?
- 모호한 문법으로부터 파싱 테이블을 구성하는 방법
- LR 파서를 실제적인 컴파일러의 구문분석기로 구현하는 내용

6.3 상향식 구문 분석

IT COOKBOOK

LR 구문 분석의 장점

- LR 구문 분석기는 모호하지 않은 문맥자유 문법으로 쓰인 모든 프로그래밍 언어에 대해 구성이 가능하다.
- LR 구문 분석 방법은 가장 일반적인 역추적이 없는 결정적인 이동-감축 구문 분석 방법이다.
- LR 구문 분석기는 입력을 왼쪽에서 오른쪽으로 검조하면서 구문 오류를 쉽게 발견할 수 있다.

LR 구문 분석의 단점

- 프로그래밍 언어에 대해 LR 구문 분석기를 직접 작성하는 일이 너무 방대한 작업

LR 구문 분석 종류 - 파싱표를 어떻게 구성하느냐에 따라

- SLR(simple LR) 구문 분석 - LR(0) 항목의 집합과 FOLLOW
- CLR(canonical LR) 구문 분석 - LR(1) 항목의 집합
- LALR(lookahead LR) 구문 분석 - LR(0) 항목의 집합과 lookahead로부터 또는 LR(1) 항목의 집합

6.3 상향식 구문 분석

IT COOKBOOK

LR 구문 분석기의 구성

- 이동-감축 구문 분석기와 마찬가지로 구동 프로그램 및 action과 GOTO 두 부분을 포함한 파싱표로 구성되고 파싱표의 행동도 이동 -감축 구문 분석기와 똑같다. 구동 프로그램은 LR 구문 분석기가 모두 같지만 파싱표는 구문 분석기에 따라 다르다.

LR 구문 분석에 대한 장단점

- SLR 방법은 구현하기가 쉽지만 강력하지 못하며, CLR 방법은 가장 강력하지만 구현하기가 매우 어렵다. LALR 방법은 SLR과 CLR의 중간 형태로 강력함은 CLR과 유사하고 파싱표의 크기는 SLR 방법으로 구성 가능.
- 이 세 가지 방법의 부분집합 관계는 [그림 6-4]와 같다.

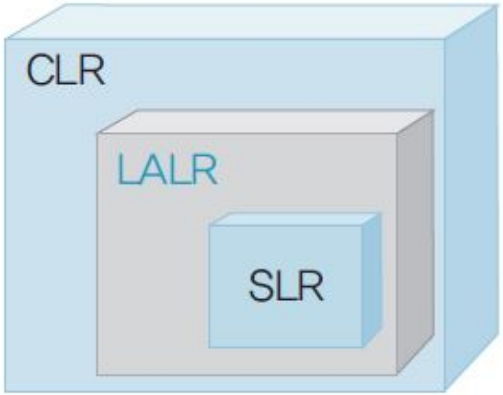

그림 6-4 LR 구문 분석의 부분집합 관계

6.3 상향식 구문 분석

IT COOKBOOK

■ LR 파싱표 행동 구성

기호 / 상태 번호	ACTION 표	GOTO 표
	$V_T \cup \{\$\}$	V_N
0	이동(shift)	상태 번호
1	감축(reduce)	
2	수락(accept)	
:	에러(error)	

■ LR 파싱표의 4가지 행동

1. 이동 : ACTION[S_m, a_i] = 이동 S

   S_m 상태에서 입력 기호 a_i를 본 행동이 이동 S라는 것은 "입력 기호를 스택으로 이동하고 다음 상태를 S로 만들기 위해 S도 stack에 넣는다."는 의미

2. 감축 : ACTION[S_m, a_i] = 감축 A $\rightarrow \alpha$

   S_m 상태에서 입력 기호 a_i를 본 행동이 감축 A $\rightarrow \alpha$ 라는 의미는 스택에서 핸들 α 를 제거 하고 스택에 생성 규칙의 오른쪽 부분을 다음 상태와 함께 스택에 넣는다.

3. 수락 : ACTION[S_m, a_i] = 수락

   S_m 상태에서 입력 기호 a_i를 본 행동이 수락이라면 주어진 문자열이 올바른 문자열이라는 의미이며 parsing을 끝낸다.

4. 에러 : ACTION[S_m, a_i] = 에러

   S_m 상태에서 입력 기호 a_i를 본 행동이 에러라면 입력 문자열이 틀린 문자열이라는 의미이며 일반적으로 에러 처리 루틴을 불러 처리

■ [정의 6-10] LR(k) 문법

- LR(k) 문법은 모든 항목(entry)에 대해 유일하게 정의되는 파싱표를 만들 수 있는 문법을 말한다. 여기서 k를 lookahead의 길이이다.

■ [정의 6-11] 증가 문법

- $G = (V_N, V_T, P, S)$에서 G에 추가된 문법으로 $G' = (V_N \cup \{S'\}, V_T, P \cup \{S' \rightarrow S\}, S')$ 이다.
  - S' : 새로운 시작 기호
  - $S' \rightarrow S$: 추가된 생성 규칙
  - 생성 규칙$S' \rightarrow S$를 두는 이유 - 주어진 문장이 안전하게 수락되게 하기 위해 사용

[정의 6-12] LR(0) 항목

- LR(0) 항목은 생성 규칙의 오른쪽에 점 기호(dot symbol)를 가진 생성 규칙이다

[정의 6-13] LR(0) 항목의 세가지 종류

- LR(0) 항목 [A → α•β]에서 α≠ε이거나 A = S′(S′는 증가 문법의 시작 기호)이면 이 항목을 **커널(kernel) 항목**
- LR(0) 항목 [A →•α]와 같이 점 기호가 생성 규칙 처음에 있는 항목을 **클로저(closure) 항목**
- LR(0) 항목 [A → α•]와 같이 생성 규칙 끝에 점 기호가 있는 항목을 **감축 항목**

정규 항목 집합(canonical collection)이라 불리는, 어떤 문법에 대한 LR(0) 항목으로 구성된 집합은 LR 파싱표를 구성하는 데 반드시 필요하다. 주어진 문법에 대한 정규 항목 집합을 구성하려면 하나의 증가 문법과 2개의 함수 CLOSURE, GOTO를 정의해야 한다.

[정의 6-14] CLOSURE(I)를 구하는 규칙

- I가 문법 G에 대한 LR(0) 항목으로 구성된 집합이면 CLOSURE(I)는 다음 두 가지 규칙에 따라 구한다.
  - 초기에 I에 속한 모든 LR(0) 항목을 CLOSURE(I)에 추가한다.
  - [A → α•Bβ]가 CLOSURE(I)에 포함되어 있고 B → γ의 생성 규칙이 존재하는 경우, B →•γ가 CLOSURE(I)에 포함되어 있지 않으면 이 항목을 CLOSURE(I)에 추가한다. 이러한 규칙은 새로운 LR(0) 항목이 CLOSURE(I)에 더 이상 추가될 수 없을 때까지 반복적으로 계속 적용된다.
  - **즉,** CLOSURE(I) = I ∪ {[B → .γ] | [A → α.Bβ] ∈ CLOSUR(I), B→γ ∈P}

CLOSURE(I)를 구하는 알고리즘 [알고리즘 6-4]와 같다.

[정의 6-15] GOTO 함수

- LR(0) 항목의 GOTO 함수는 I가 항목의 집합이고 X ∈ V이면 GOTO(I, X) = CLOSURE({[A → αX•β] | [A → α•Xβ ∈ I})이다.

[정의 6-16] LR(0) 항목 집합의 정규 항목 집합

- LR(0) 항목 집합의 정규 항목 집합은 C_0으로 표기하며, $C_0 = \{CLOSURE(\{[S' \rightarrow \bullet S]\})\} \cup \{GOTO(I, X) \mid I \in C_0, X \in V\}$이다.

SLR 구문 분석

- SLR 구문 분석은 LR 구문 분석 방법 중 가장 간단하게 구현할 수 있는 방법으로, LR(0) 항목 집합과 FOLLOW를 이용하여 SLR 파싱표를 만든다. SLR 파싱표는 LR(0) 항목의 정규 항목 집합으로부터 이동과 GOTO 행동을 구하고, 생성 규칙에 있는 논터미널 기호의 FOLLOW를 가지고 감축을 결정한다. SLR 구문 분석기의 파싱표 구성은 [알고리즘 6-5]와 같다.

[알고리즘 6-5] SLR 파싱표 구성

- [입력] 문법 $G = (V_N, V_T, P, S)$
- [출력] 증가 문법 G′에 대한 SLR 파싱표
- [방법] ❶ G′를 위한 LR(0) 항목 집합의 정규 항목 집합 C0 = {I0, I1, ..., In}을 구성한다. ❷ 상태 i는 Ii에서 구성되었다. 상태 i를 위한 구문 분석 동작은 다음과 같이 결정된다. • 만약 LR(0) 항목 [A → α•aβ] ∈ Ii, GOTO(Ii, a) = Ij이면 action 표의 M[i, a] = 이동 j • 만약 LR(0) 항목 [A → α•] ∈ Ii이면 논터미널 기호 A의 FOLLOW(A)에 속한 모든 기호 a에 대해 action 표의 M[i, a] = 감축 A → α •만약 LR(0) 항목 [S′ → S•] ∈ Ii이면 action 표의 M[i, $] = 수락 ❸ 만약 LR(0) 항목 [A → α•Bβ] ∈ Ii이고 GOTO(Ii, B) = Ij이면 GOTO 표의 M[i, B] = j ❹ 이렇게 해서 정의되지 않은 파싱표의 항목들은 모두 에러가 되며, 첨가된 생성 규칙의 LR(0) 항목 [S′ →•S]를 포함하는 상태가 시작 상태이다

6.3 상향식 구문 분석

IT COOKBOOK

- **SLR 구문 분석기를 만드는 과정**

① 문법이 주어진다.

② 증가 문법을 만든다.

③ LR(0) 항목 집합의 정규 항목 집합을 구한다.

④ 감축 행동을 결정하기 위해 FOLLOW를 계산한다.

⑤ SLR 파싱표를 작성한다.

IT COOKBOOK

[알고리즘 6-6] LR 구문 분석 알고리즘

- [입력] 입력 문자열 w와 LR 파싱표
- [출력] 만약 w가 L(G) 안에 있으면 w에 대한 파스 트리를 생성하고, 그렇지 않으면 에러
- [방법] 구문 분석기는 스택의 초기 상태인 S0을 가지고 있다.

```
begin
  ip는 w$의 처음 기호를 나타낸다.
  repeat
    s 는 stack의 톱 상태
    a 는 ip에 의해 지적된 기호;
    if action [s, a] = 이동 s'
      then begin a를 스택에 삽입하고 그 다음에 s'를 스택에 삽입한다.
                 ip 가 다음 입력 기호로 이동한다.
           end
      else if action [s, a] = 감축 s'(생성 A → β)
             then begin |β|개의 기호를 스택에서 삭제한다.
                        A 를 스택에 삽입한다.
                        GOTO[s', A] = n이라면 n을 스택에 삽입한다.
                  end
             else if action [s, a] = 수락
                     then return
                     else 에러
  until (ip = $이고 스택의 톱 = $)
end.
```

CLR 구문 분석

- SLR 방법에서 파싱표를 작성할 때 FOLLOW에 속하는 기호에 대해 감축 행동을 만들었다. 그러나 어떤 상태에서는 감축 항목 [A → α•]에 대한 FOLLOW(A)에는 속하지만 그 상태에서 A 다음에 나올 수 없는 기호가 존재한다. 반면에 한 상태에서 어떤 논터미널 기호 다음에 나올 수 있는 정확한 터미널 기호를 lookahead라고 하는데, 이를 이용하는 방법이 CLR 구문 분석이다.
- LR(0) 항목에서 lookahead 정보를 첨가한 것이 LR(1) 항목이며, 이는 [정의 6-17]과 같다.

[정의 6-17] LR(1) 항목

- LR(1) 항목은 [A → α•β, a] 형태이며, 여기서 A → αβ ∈ P이고 a ∈ {V_T ∪ $}이다.
  - 첫 번째 부분인 A → α•β를 코어(core)라 하며, 이는 LR(0) 항목과 같은 의미이다.
  - 두 번째 부분인 a를 lookahead라 하며, 이는 감축 항목일 때 그 기호에 대해 감축 행동을 하라는 것이다.

6.3 상향식 구문 분석

IT COOKBOOK

CLR 파싱표 작성

- LR(1) 항목 집합의 정규 항목 집합으로 구성
  - CLOSURE 함수와 GOTO 함수가 필요
  - GOTO 함수는 LR(0) 항목 집합의 정규 항목 집합을 구할 때와 같지만, CLOSURE 함수는 lookahead 정보 때문에 약간 수정해야 함

[정의 6-18] LR(1) 항목에서의 CLOSURE 함수 구하기

$$\mathbf{CLOSURE(I) = I \cup \{[B \rightarrow .\gamma,\ b] \mid [A \rightarrow \alpha.B\beta,\ a] \in CLOSURE(I),\ B \rightarrow .\gamma \in P,\ b \in FIRST(\beta a)\}}$$

- LR(1) 항목 집합의 CLOSURE는 LR(0) 항목 집합의 CLOSURE와 유사하며 lookahead를 구해서 첨가 하는 것만이 차이점
  - $[A \rightarrow \alpha.B\beta,\ a]$에서 마크 기호 B다음에 오는 β의 FIRST가 항목 $[B \rightarrow .\gamma]$의 lookahead
  - 만일 β가 ε을 유도 할 수 있으면, 항목 $[A \rightarrow \alpha.B\beta]$의 lookahead인 a도 lookahead가 된다.
    - 항목 $[B \rightarrow .\gamma]$의 lookahead는 βa의 FIRST가 된다.

6.3 상향식 구문 분석

IT COOKBOOK

- LR(1) 항목의 정규 항목 집합을 만드는 방법은 SLR에서 LR(0) 항목의 정규 항목 집합을 구하는 방법과 동일하며, 이는 [알고리즘 6-7]과 같다.

LALR 구문 분석

- LALR 구문 분석 방법은 lookahead 정보를 이용하기 때문에 lookahead LR 방법이라고도 한다.
  - SLR 방법보다 훨씬 강력하고, 파싱표의 크기가 CLR 방법보다 상당히 작으면서도 프로그래밍 언어의 구문 구조를 대부분 편리하게 표현할 수 있다는 것이 장점이다. 파싱표의 크기를 비교해보면 SLR 파싱표와 LALR 파싱표는 주어진 문법에 대해 항상 동일한 개수의 상태로 만들 수 있으며, C 언어의 경우 전형적으로 수백 개 정도이다. 반면에 CLR 파싱표는 동일한 규모의 언어에 대해 전형적으로 수천 개의 상태이다. 그러므로 CLR 파싱표보다 SLR 파싱표나 LALR 파싱표를 구성하는 것이 훨씬 더 쉽고 경제적이다.
  - 또한 모호하지 않은 문맥자유 문법으로 표현된 거의 모든 언어를 인식할 수 있으며, 에러를 초기에 탐지할 수 있다. 그러나 LALR 방법은 LR(0) 항목의 집합을 구한 후 파싱표를 만들기 위해 감축 항목이 있는 각 상태에서 lookahead 기호를 구하는 데 상당한 시간과 노력이 소요된다.
  - LALR 파싱표를 구성하는 방법은 두 가지가 있다. 파싱표를 구성하기가 쉽지만 파싱표의 크기가 커지는 LR(1)을 가지고 구성하는 방법과 효율적으로 파싱표를 구성할 수 있는 LR(0)과 lookahead를 가지고 구성하는 방법이 그것이다. 첫 번째 방법은 먼저 LR(1) 항목을 가지고 파싱표를 구성하는 방법으로 같은 코어를 가진 LR(1) 항목을 묶어서 하나의 상태로 구성하는 것이며, lookahead는 LR(1) 항목의 lookahead를 합한 것이다.

[정의 6-19] lookahead의 정의

- 상태 p에서 LR(0) 항목 [A→$\alpha.\beta$] 의 lookahead는 LA(p, [A→$\alpha.\beta$])로 표기하며 p 상태에서 A 다음에 나올 수 있는 터미널 기호의 집합이다.

$$\mathbf{LA(p, [A \rightarrow \alpha.\beta]) = \{a \mid a \in FIRST(\delta),\ S' \overset{*}{\Rightarrow} \gamma A \delta \Rightarrow \gamma\alpha\beta\,\delta,\ \gamma\alpha \text{ accesses } p\}.}$$

- $\gamma\alpha$가 상태 p를 access한다는 것은 시작 상태로부터 $\gamma\alpha$만큼 보고 상태 p로 이동을 하였다는 의미

모호한 문법의 사용과 에러 처리 루틴

- 모호한 문법은 언어의 명세에서 매우 유용하다. 표현식과 같은 언어 구조에 대해 모호한 문법은 동등한 모호하지 않은 문법보다 더 짧고 더 자연스러운 명세를 제공한다. 즉 모호한 문법은 동등한 모호하지 않은 문법보다 문법의 표현이 간단하며, 사람들이 사용하기에 훨씬 자연스러운 문법이다.
- 모호한 문법을 가지고 파싱표를 작성하면 항상 이동 -감축 충돌이나 감축 -감축 충돌을 야기한다.
- 이러한 충돌을 없애는 두 가지 방법이 있다. 첫 번째 방법은 앞에서 다루었던 모호한 문법을 동등한 의미를 가진 모호하지 않은 문법으로 변환하는 것이다. 두 번째 방법은 모호한 문법을 가지고 파싱표를 작성한 다음 파싱표에서 발생하는 충돌을 제거하는 것.
- 두 가지의 모호한 문법을 통해 파싱표에 나타난 충돌을 제거하는 방법을 도입. 첫 번째는 산술식에서 연산자 우선순위와 결합 법칙을 이용하여 충돌을 제거하는 것이고, 두 번째는 현수 else라고 하는 모호한 문법이다.
- 먼저 산술식에서 연산자 우선순위와 결합 법칙을 이용하여 충돌을 제거하는 방법을 살펴보자. 충돌은 이동 -감축 충돌과 감축 -감축 충돌이 있는데, 이동 -감축 충돌의 경우 감축되는 생성 규칙과 입력 기호의 우선순위를 비교하여 생성 규칙의 우선순위가 높으면 감축을 선택 하고, 그렇지 않으면 이동을 선택한다. 생성 규칙의 우선순위는 그 생성 규칙 내에 있는 터미널 기호로 결정할 수 있다. 반면에 감축 -감축 충돌은 감축되는 생성 규칙의 우선순위를 비교 하여 우선순위가 높은 쪽으로 감축하면 된다

6.4 모호한 문법의 사용과 에러 처리 루틴

IT COOKBOOK

■ 충돌을 제거하는 방법

- 이동-감축 충돌인 경우 충돌을 해결하는 방법
  - 감축되는 생성규칙과 입력 기호의 우선 순위를 비교하여 결정
    - 생성규칙의 순위가 높으면 감축을 선택, 그렇지 않으면 이동을 선택
  - 같은 순서인 경우에는 결합 법칙을 이용
    - 좌측 결합을 만족하면 감축, 우측 결합을 만족하면 이동을 선택
- 감축-감축 충돌인 경우 충돌을 해결하는 방법
  - 감축되는 생성규칙들의 순위를 비교하여 어느 생성규칙으로 감축할 것인가를 결정하는데 우선 순위가 높은 것을 선택
    - 생성규칙의 순위는 그 생성규칙 내에 있는 터미널 기호로 결정 가능